Ejercicio JPA - CRUD de una tabla

1. Descripción general del ejercicio

El objetivo de esta práctica es **desarrollar una aplicación web completa** utilizando el framework **Spring Boot**, conectada a una base de datos **MySQL**, que permita realizar operaciones **CRUD (Crear, Leer, Actualizar y Eliminar)** sobre una tabla determinada.

Cada alumno trabajará con **una de las siguientes entidades**, que representan diferentes contextos de aplicación:

- **Citas Telefónicas:** gestión de citas con clientes o usuarios.
- **Socios de un Club:** gestión de los socios registrados en una organización o asociación.
- **Publicaciones (Libros y Revistas):** gestión de los materiales disponibles en una biblioteca.

A pesar de las diferencias en los datos gestionados, **la estructura del proyecto y las capas de la aplicación serán idénticas para todos los alumnos**, aplicando el patrón de arquitectura **MVC (Modelo – Vista – Controlador)**.

El proyecto deberá incluir las siguientes capas:

- **Modelo:** representación de la tabla mediante una clase de entidad JPA con sus anotaciones y campos correspondientes.
- **Repositorio:** interfaz para el acceso a los datos que extienda `JpaRepository`.
- **Servicio:** clase intermedia que gestione la lógica de negocio y las operaciones del repositorio.
- **Controlador:** clase encargada de gestionar las peticiones web y devolver las vistas adecuadas.
- **Vistas (Thymeleaf):** páginas HTML con formularios y listados para interactuar con el usuario.

El objetivo final es disponer de una aplicación funcional que permita:

- Mostrar un listado de registros de la tabla.
- Crear nuevos registros mediante un formulario.
- Editar registros existentes.
- Eliminar registros de la base de datos.

2. Configuración del proyecto

Cada alumno debe crear un proyecto **Spring Boot con Maven** utilizando **Visual Studio Code**.
El nombre del proyecto y de la base de datos dependerá de la entidad asignada:

Entidad	Nombre del proyecto	Base de datos
Citas Telefónicas	`crud_citas`	`agenda`
Socios de un Club	`crud_socios`	`club`
Publicaciones (Libros y Revistas)	`crud_publicaciones`	`biblioteca`

El proyecto debe incluir las dependencias necesarias para:

- El **acceso a datos** mediante **Spring Data JPA** y **Hibernate**.
- La **gestión del servidor web** y las peticiones con **Spring Web**.
- La **creación de vistas dinámicas** con **Thymeleaf**.
- La **conexión a la base de datos MySQL**.

Cada alumno deberá configurar manualmente la conexión a su base de datos en el archivo de propiedades del proyecto, siguiendo las indicaciones vistas en los apuntes del tema **“Acceso a almacenes de datos”**.

También deberá verificar que el proyecto se ejecuta correctamente y que la aplicación se inicia sin errores antes de continuar con la implementación del modelo y las capas restantes.

3. Creación de la base de datos y de la tabla en MySQL

Cada alumno debe crear su propia base de datos en **MySQL** utilizando **phpMyAdmin**, según la entidad que tenga asignada.
A continuación, se indican los nombres de las bases de datos y las sentencias SQL necesarias para crear las tablas correspondientes.

Después de crear la tabla, se recomienda insertar algunos registros de ejemplo para realizar las pruebas del CRUD.

3.1 Citas Telefónicas

Base de datos: `agenda`
Tabla: `citas`

```
CREATE TABLE citas (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  telefono VARCHAR(15) NOT NULL,
  motivo VARCHAR(200) NOT NULL,
  fecha_cita DATE NOT NULL,
  hora_cita TIME NOT NULL,
  confirmada BOOLEAN DEFAULT FALSE,
  observaciones TEXT
);
```

Ejemplo de inserción de datos:

```
INSERT INTO citas (nombre, telefono, motivo, fecha_cita, hora_cita,
confirmada) VALUES
('Laura Gómez', '678123456', 'Consulta comercial', '2025-02-15',
'10:30:00', TRUE),
('Carlos Ruiz', '612987321', 'Revisión de contrato', '2025-02-18',
'12:00:00', FALSE);
```

3.2 Socios de un Club

Base de datos: `club`
Tabla: `socios`

```
CREATE TABLE socios (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  apellidos VARCHAR(100) NOT NULL,
  dni VARCHAR(10) NOT NULL UNIQUE,
  telefono VARCHAR(15),
  email VARCHAR(100),
  cuota_pagada BOOLEAN DEFAULT FALSE,
  fecha_alta DATE NOT NULL
);
```

Ejemplo de inserción de datos:

```
INSERT INTO socios (nombre, apellidos, dni, telefono, email,
cuota_pagada, fecha_alta) VALUES
('Ana Pérez', 'Martínez', '12345678A', '666112233',
'ana.perez@email.com', TRUE, '2024-11-10'),
('David López', 'García', '87654321B', '677445566',
'david.lopez@email.com', FALSE, '2024-12-05');
```

3.3 Publicaciones (Libros y Revistas)

Base de datos: `biblioteca`
Tabla: `publicaciones`

```
CREATE TABLE publicaciones (
  id INT AUTO_INCREMENT PRIMARY KEY,
  titulo VARCHAR(200) NOT NULL,
  autor VARCHAR(100) NOT NULL,
  editorial VARCHAR(100),
  anio_publicacion YEAR,
  tipo VARCHAR(50) NOT NULL,
  disponible BOOLEAN DEFAULT TRUE,
  ubicacion VARCHAR(50)
);
```

Ejemplo de inserción de datos:

```
INSERT INTO publicaciones (titulo, autor, editorial,
anio_publicacion, tipo, disponible, ubicacion) VALUES
```

```
('Introducción a Spring Boot', 'Ana García', 'TechPress', 2023,
'Libro', TRUE, 'Estantería A3'),
('Revista Java Hoy', 'Luis Pérez', 'SoftMag', 2025, 'Revista',
FALSE, 'Estantería C2');
```

- **Tareas a realizar:**

Inserta otros dos registros siguiendo el modelo dado.

Inventate los datos.

La tabla tiene que tener 4 registros.

4. Entidad JPA (Modelo)

Cada alumno deberá crear una **clase de entidad JPA** que represente la tabla correspondiente a su proyecto.
Esta clase formará parte de la **capa de modelo** dentro de la arquitectura MVC y será la encargada de reflejar en el código los campos definidos en la tabla de la base de datos.

Indicaciones generales

La clase debe estar situada dentro de un paquete con nombre similar a:
`com.ejemplo.crud_citas.modelos`
`com.ejemplo.crud_socios.modelos`
`com.ejemplo.crud_publicaciones.modelos`

(según el proyecto correspondiente).

1. La clase debe incluir:
    - Las **anotaciones JPA necesarias** (`@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Column`, etc.).
    - Los **atributos** correspondientes a las columnas de la tabla.
    - Un **constructor vacío** (requerido por JPA).
    - Un **constructor con parámetros** (para crear objetos fácilmente).
    - Los **métodos getter y setter** para todos los atributos.
    - El **método `toString()`** para mostrar los datos de forma legible durante las pruebas.
2. El nombre de la clase debe corresponder con la entidad gestionada:
    - `Cita.java` para el proyecto `crud_citas`
    - `Socio.java` para el proyecto `crud_socios`
    - `Publicacion.java` para el proyecto `crud_publicaciones`
3. Los tipos de datos de los atributos deben corresponder con los tipos SQL definidos en la tabla:
    - `VARCHAR` → `String`
    - `DATE` → `LocalDate`
    - `TIME` → `LocalTime`
    - `BOOLEAN` → `boolean o Boolean`
    - `YEAR` → `int` o `Integer`
    - `TEXT` → `String`

4. Se recomienda incluir **comentarios explicativos** en el código para describir el propósito de cada atributo y de cada anotación.

Objetivo de este punto

Al finalizar este apartado, el alumno debe haber creado una clase Java que:

- Mapea correctamente la tabla de la base de datos.
- Permite a Spring Data JPA reconocerla como una entidad gestionada.
- Sirve de base para las operaciones CRUD que se desarrollarán en los apartados siguientes.

5. Repositorio (Acceso a datos)

En este apartado, cada alumno debe crear una **interfaz de repositorio** que se encargue de gestionar las operaciones de acceso a la base de datos relacionadas con la entidad de su proyecto.

Esta capa forma parte del **modelo** en la arquitectura **MVC** y actúa como un intermediario entre las entidades JPA y el resto de la aplicación.

Indicaciones generales

Crear un nuevo paquete llamado:
`com.ejemplo.crud_citas.repositorios`
`com.ejemplo.crud_socios.repositorios`
`com.ejemplo.crud_publicaciones.repositorios`

(según el proyecto asignado)

1. Dentro de ese paquete, crear una **interfaz** con el nombre adecuado:
    - `CitaRepository`
    - `SocioRepository`
    - `PublicacionRepository`

2. Esta interfaz debe **extender la interfaz genérica `JpaRepository`**, indicando:
    - El tipo de entidad (por ejemplo, `Cita`, `Socio` o `Publicacion`).
    - El tipo de dato de la clave primaria (`Long` o `Integer`).

3. Gracias a `JpaRepository`, se dispondrá automáticamente de los principales métodos CRUD:
    - `findAll()` → Listar todos los registros.
    - `findById()` → Buscar un registro por su ID.
    - `save()` → Insertar o actualizar un registro.
    - `deleteById()` → Eliminar un registro.

Objetivo de este punto

Al finalizar este apartado, el alumno debe tener un **repositorio funcional** que:

- Permita realizar operaciones básicas de persistencia (CRUD).

6. Servicio (Lógica de negocio)

La capa de **servicio** actúa como un **intermediario** entre el **repositorio** y el **controlador**, y su función principal es gestionar la **lógica de negocio** de la aplicación.
En esta capa se definen los métodos que manejarán los datos obtenidos del repositorio antes de ser enviados al controlador.

Indicaciones generales

Crear un nuevo paquete con el nombre:

`com.ejemplo.crud_citas.servicios`
`com.ejemplo.crud_socios.servicios`
`com.ejemplo.crud_publicaciones.servicios`
(según el proyecto correspondiente.)

1. Dentro del paquete, crear una clase llamada:
    - `CitaService`
    - `SocioService`
    - `PublicacionService`
2. Esta clase debe estar anotada con **`@Service`**, para que Spring la detecte automáticamente como un componente de servicio.
3. En su interior se debe **inyectar** el repositorio correspondiente mediante el constructor (o con la anotación `@Autowired`.)
4. Implementar en esta clase los **métodos de negocio** que permitan:
    - Listar todos los registros (método `listarTodos`).
    - Buscar un registro por su identificador (`buscarPorId`).
    - Guardar un nuevo registro o actualizar uno existente (`guardar`).
    - Eliminar un registro (`eliminar`).
5. Todos los métodos deben estar **claramente comentados**, explicando qué tarea realiza cada uno.

Objetivo de este punto

Al finalizar este apartado, el alumno debe haber creado una clase de servicio que:

- Centraliza la lógica de negocio de la aplicación.
- Se comunica directamente con el repositorio para realizar las operaciones CRUD.
- Permite que el controlador se mantenga limpio y organizado, delegando en el servicio el trabajo con los datos.

7. Controlador (Capa web)

El **controlador** representa la parte de la aplicación encargada de **recibir las peticiones del usuario**, **comunicarse con el servicio** para obtener o modificar los datos, y **devolver la vista correspondiente**.
Forma parte de la **C** del patrón **MVC (Modelo – Vista – Controlador)**.

Indicaciones generales

Crear un nuevo paquete con el nombre:

`com.ejemplo.crud_citas.controladoras`
`com.ejemplo.crud_socios.controladoras`
`com.ejemplo.crud_publicaciones.controladoras`
(según el proyecto correspondiente.)

1. Dentro del paquete, crear una clase llamada:
    - `CitaController`
    - `SocioController`
    - `PublicacionController`

2. La clase debe estar anotada con **`@Controller`**, indicando que se trata de un componente de presentación que gestionará las rutas del navegador.

3. Inyectar la clase de servicio correspondiente (por ejemplo, `CitaService`, `SocioService`, `PublicacionService`) mediante el constructor o con la anotación `@Autowired`.

4. Definir las **rutas o endpoints** necesarios para cada operación del CRUD, usando las anotaciones adecuadas de Spring MVC:
    - `@GetMapping` → para mostrar páginas o listados.
    - `@PostMapping` → para procesar formularios de creación o edición.
    - `@PathVariable` → para capturar identificadores en la URL.
    - `@ModelAttribute` → para vincular los datos del formulario con el objeto de la entidad.

5. Las operaciones mínimas que deben implementarse son:

   Cambiar **entidad** por una ruta sobre el proyecto asignado: **citas, socios** o **publicaciones**

Operación	Método	Descripción
Listar registros	`GET /entidad`	Muestra todos los registros en una vista de lista.
Nuevo registro	`GET /entidad/nuevo`	Muestra el formulario para añadir un nuevo registro.
Guardar registro	`POST /entidad/guardar`	Envía los datos del formulario para crear o actualizar un registro.
Editar registro	`GET /entidad/editar/{id}`	Muestra el formulario de edición para un registro existente.
Eliminar registro	`GET /entidad/eliminar/{id}`	Elimina el registro seleccionado y redirige al listado.

6. El controlador debe devolver siempre el nombre de la **plantilla Thymeleaf** que se renderizará como vista, y enviar los datos necesarios al modelo (`Model model`).

7. Se deben incluir **comentarios explicativos** sobre el propósito de cada método y sobre las rutas configuradas.

Objetivo de este punto

Al finalizar este apartado, el alumno debe haber creado un **controlador funcional** que:

- Recibe las peticiones del navegador.
- Utiliza los métodos del servicio para manipular los datos.
- Devuelve las vistas correspondientes a cada operación del CRUD.
- Sirve de enlace entre la lógica de negocio y la interfaz de usuario.

8. Vistas (Thymeleaf y fragmentos)

Las **vistas** son las páginas HTML que el usuario visualizará en el navegador.
En esta práctica se crearán utilizando el motor de plantillas **Thymeleaf**, que permite **mostrar datos dinámicos** provenientes del controlador y enlazarlos fácilmente con los objetos del modelo.

Indicaciones generales

Las vistas deben ubicarse en la carpeta:

```
src/main/resources/templates/
```

1. Cada proyecto debe incluir, como mínimo, las siguientes plantillas:

Vista	Propósito
`lista.html`	Muestra todos los registros almacenados en la base de datos.
`form.html`	Muestra el formulario para crear o editar un registro.
`detalle.html`	Muestra la información completa de un registro concreto.

2. Todas las vistas deben estar conectadas al **controlador** mediante las rutas configuradas, de forma que cada acción (crear, editar, eliminar) se refleje correctamente en la base de datos.

Uso de fragmentos Thymeleaf

Para mantener una estructura uniforme en todas las páginas, se deberá crear un archivo adicional llamado:

`fragmentos.html`

Este archivo contendrá **un fragmento reutilizable** con el encabezado común que compartirán todas las vistas. Dicho encabezado debe incluir:

- El título de la aplicación.
- Un menú de navegación o enlaces a las principales rutas (por ejemplo, “Inicio” y “Nuevo registro”).
- Cualquier otro elemento decorativo o informativo que se quiera mostrar en todas las páginas.

Cada vista (`lista.html`, `form.html`, etc.) deberá **insertar este fragmento** al inicio de su contenido, para que el encabezado se muestre de forma idéntica en todas ellas.

Recordatorio:
El uso de fragmentos favorece la **reutilización de código**, **facilita el mantenimiento** y **asegura coherencia visual** en toda la aplicación.

9. Prueba del CRUD completo

En este apartado, el alumno deberá comprobar que **todas las operaciones del CRUD funcionan correctamente** desde el navegador web, asegurándose de que la aplicación cumple los objetivos establecidos.

El propósito es **verificar la integración total** entre las distintas capas de la aplicación:
Modelo → Repositorio → Servicio → Controlador → Vista.

Pasos para la comprobación

1. **Ejecución del proyecto**
    - Iniciar la aplicación Spring Boot desde Visual Studio Code.
    - Acceder al navegador y abrir la dirección predeterminada del servidor local
    - Confirmar que la aplicación arranca sin errores y muestra la página principal.

2. **Listado de registros**
    - Verificar que la vista `lista.html` muestra correctamente todos los registros de la base de datos.
    - Comprobar que los datos iniciales insertados en phpMyAdmin aparecen correctamente en la tabla HTML.

3. **Creación de un nuevo registro**
    - Acceder al formulario de creación mediante el enlace o botón “Nuevo”.
    - Rellenar los campos y guardar el registro.
    - Confirmar que el nuevo registro aparece en el listado y que se ha añadido en la base de datos.

4. **Edición de un registro existente**
    - Seleccionar un registro del listado y acceder a su formulario de edición.
    - Modificar algunos valores y guardar.
    - Verificar que los cambios se reflejan tanto en la vista como en la base de datos.

5. **Visualización del detalle de un registro**
    - `Detalle.html`, acceder a ella desde el listado mediante el enlace correspondiente.
    - Comprobar que se muestran correctamente todos los campos del registro seleccionado.
    - Verificar que la página incluye un enlace o botón para volver al listado general.

6. **Eliminación de un registro**
    - Pulsar el botón o enlace “Eliminar” sobre un registro.
    - Confirmar que el registro desaparece del listado.
    - Revisar en phpMyAdmin que también ha sido eliminado de la tabla.

7. **Pruebas de navegación y diseño**
    - Asegurarse de que todos los enlaces y botones entre las distintas vistas funcionan correctamente.
    - Comprobar que el encabezado común (fragmento Thymeleaf) aparece correctamente en todas las páginas.
    - Revisar que no existen errores de redirección ni vistas en blanco.

Anexo

Una aplicación web, como el proyecto que estáis haciendo, debe llevar más funcionalidad. Por ejemplo estas caracteristicas:

1. **Validación de datos en los formularios**
    - Añadir anotaciones de validación en la entidad (`@NotBlank`, `@Size`, `@Email`, `@Min`, `@Max`, etc.) para controlar que los campos se introduzcan correctamente.
    - Mostrar mensajes de error en la vista `form.html` cuando los datos no cumplan las condiciones.
    - Aplicar las validaciones más adecuadas según la entidad:
        - En *Citas Telefónicas*: verificar que el nombre y el teléfono no estén vacíos.
        - En *Socios de un Club*: comprobar el formato del correo electrónico y del DNI.
        - En *Publicaciones*: asegurar que el título y el tipo de publicación estén informados.
2. **Mensajes de confirmación y notificaciones al usuario**
    - Mostrar mensajes visuales cuando se realicen operaciones correctamente (por ejemplo, “Registro guardado con éxito” o “Registro eliminado”).
    - También se pueden mostrar mensajes de advertencia cuando ocurra un error.
3. **Búsqueda o filtrado de registros**
    - Añadir un cuadro de búsqueda en la vista `lista.html` que permita filtrar los registros según un criterio (por nombre, autor, fecha, etc.).
    - Implementar en el controlador y el servicio un método que gestione el filtrado de resultados.
4. **Ordenación de los registros**
    - Permitir ordenar los registros del listado por una columna concreta (por ejemplo, por fecha, nombre o título).
    - Se puede hacer desde el repositorio o mediante opciones de ordenación en la vista.
5. **Paginación de resultados**
    - Si la tabla contiene muchos registros, implementar un sistema de paginación que divida el listado en varias páginas.
    - Utilizar las funcionalidades de paginación que ofrece `JpaRepository`.

6. **Diseño y estilo del proyecto**

    - Mantener una interfaz coherente y agradable para el usuario.
    - Incorporar iconos o colores para identificar las acciones (editar, eliminar, ver, etc.).

7. **Exportación o importación de datos (opcional avanzado)**

    - Implementar la exportación de los registros a un archivo CSV o PDF.
    - Añadir la posibilidad de importar datos desde un archivo externo.

Asignación de proyecto

Alumno	Proyecto
Alberto	Citas
Francisco	Socios
Diego	Publicaciones
Hugo	Citas
Dario	Socios
Iker	Publicaciones
Iván L	Socios
Gian Carlos	Publicaciones
Angel	Citas
Alejandro F	Socios
Daniel	Publicaciones
Carlos	Citas
Alvaro	Socios
Alejandro S	Publicaciones
Mario Martos	Socios
Ivan E	Publicaciones
Leyi	Citas
Andrés	Socios
Mario Mateo	Publicaciones
Alex	Citas